
JobTracker Hub

User Guide & Workflow Playbook

What it is. JobTracker Hub is a local-first dashboard for the job-application documents you already keep on disk. It builds a searchable application index, gives you a real pipeline and triage workflow, and keeps the underlying files under your control.

Runs locally	FastAPI web UI; no account required.
Data model	Your existing folders + two local SQLite databases.
Privacy posture	Binds to localhost; no cloud service is required by the application.
License	MIT (see repository LICENSE).

Repository: JobTracker Hub — this guide lives in `docs/` alongside the code it documents.

Guide version: 1.6 | **Updated:** August 26, 2026

Layout direction informed by Overleaf's *Technical Document Template*, adapted for JobTracker Hub.

Contents

1	Start Here	2
2	How Your Folder Becomes a Tracker	4
3	First Build: What to Expect	5
4	The Web Dashboard	5
5	Working With Documents	8
6	Command Palette and Keyboard Workflow	8
7	Multiple Trackers and Workspaces	8
8	Privacy and Local Data	10
9	A Recommended Daily Workflow	10
10	Troubleshooting	10
11	Repository Map for Contributors	11
12	Repository Safety Notes	12
13	License	12

1 Start Here

1.1 What JobTracker Hub actually does

JobTracker Hub sits one level inside a folder that already contains your job-search material. It indexes the files on disk, infers useful metadata from folder and filename conventions, and presents the result through a local dashboard.

The important design choice is that the application does *not* require you to migrate your existing documents into a hosted database. Your tracker remains a normal folder you can inspect in Finder or Explorer.

The core mental model: your filesystem is the source of truth. `jobtracker.db` is a rebuildable index, while `overrides.db` holds the durable choices you make in the app, such as status overrides, notes, dates, snoozes, archives, and company merges.

1.2 Quick start

The repository includes a fake `sample-tracker/` so you can verify the installation before pointing the application at real documents.

1. Copy `sample-tracker/` to a new folder, for example `my-tracker/`.
2. Copy the repository's `_app/` directory into `my-tracker/`.
3. Create a virtual environment and install the dependencies.
4. Start the recommended web application.
5. Open <http://127.0.0.1:8000> and choose **Build index** the first time.

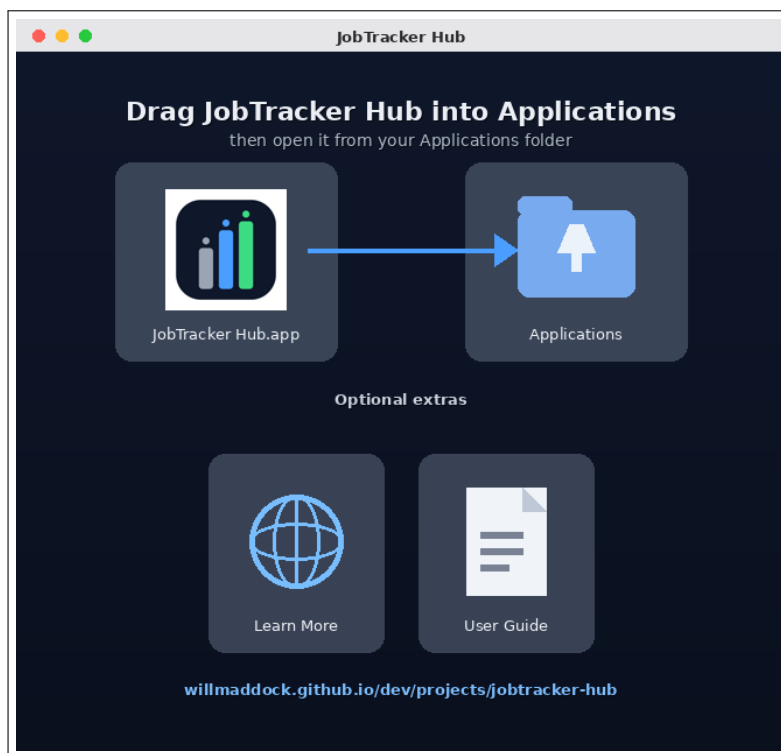
```
cp -r sample-tracker my-tracker
cp -r _app my-tracker/_app
cd my-tracker/_app
python3 -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt
uvicorn api:app --reload --port 8000
```

The frontend's requests are relative to that origin, so open the app at <http://127.0.0.1:8000> rather than opening `frontend/index.html` directly as a file — a bare file won't be able to reach the API.

1.3 Installing the macOS desktop app

If you were given a `JobTracker Hub.dmg` instead of the repository, you don't need Python, a virtual environment, or a terminal at all — the desktop build packages the backend and frontend together into a single double-clickable app.

1. Double-click `JobTracker Hub.dmg` to mount it. A window opens showing the **JobTracker Hub** icon and an **Applications** shortcut, connected by an arrow.
2. Drag the **JobTracker Hub** icon onto the **Applications** shortcut to install it.
3. Eject the mounted disk image (the **JobTracker Hub** entry under **Locations** in Finder's sidebar), then open the app from your **Applications** folder as usual.



The installer window: drag the app icon onto the Applications shortcut to install. Two optional shortcuts sit below it – **Learn More** and **User Guide**.

Below the main drag-and-drop row, the same window has two optional shortcuts that don't affect the install itself:

- **Learn More** opens the project's portfolio page in your browser – it's an Internet shortcut, not part of the app.
- **User Guide** is this document. Double-click it to open the PDF directly from the install window, before the app is even copied to **Applications**.

First launch. The very first time you open the app, it asks you to choose a folder — this becomes your tracker root, exactly like the **my-tracker/** folder in the command-line quick start above. Pick an empty folder for a fresh tracker, or an existing one if you're migrating from a manual install. The app remembers this choice and opens straight to your dashboard on every future launch.

"Not Opened" warning on first launch. This is normal for an app downloaded outside the App Store and not notarized by Apple — it is not a sign anything is actually wrong. The very first time you open **JobTracker Hub**, macOS shows a dialog titled "**JobTracker Hub.app** Not Opened" with two buttons: **Move to Trash** and **Done**. There is no **Open Anyway** button in this first dialog, so:

1. Click **Done** — *not* **Move to Trash**.
2. In **Applications**, right-click (or Control-click) **JobTracker Hub** and choose **Open** from the menu. A second dialog appears; this one has an **Open** button — click it.
3. If right-clicking doesn't offer **Open**, go to **System Settings** > **Privacy & Security**, scroll to the **Security** section, and click **Open Anyway** next to the message about **JobTracker Hub** being blocked.

You only need to do this once — macOS remembers your choice for future launches.

Unlike the command-line install, the desktop app writes its data to a per-user Application Support folder rather than next to the code, since the installed `.app` bundle itself is read-only:

```
~/Library/Application Support/JobTracker Hub
```

See Section 10 if the app doesn't open, or opens to a blank window.

1.4 Requirements

- Python 3.9 or newer.
- A modern web browser for the web UI.
- A local filesystem containing the documents you want to track.

1.5 Running the app

JobTracker runs as a single FastAPI process, `uvicorn api:app -port 8000`, which both exposes the backend as JSON and serves the frontend (`frontend/index.html`) itself. Open `http://127.0.0.1:8000` in your browser — the frontend's requests are relative to that origin, so it needs to be loaded from the running server rather than opened as a bare file. This gets you the full feature set: the dark master-detail UI, Kanban, command palette, document operations, and workspaces.

2 How Your Folder Becomes a Tracker

2.1 The recommended layout

The application expects `_app/` to live directly inside the folder it should index:

```
MyTracker/
|-- Applications/
|   |-- Acme Robotics/
|   |   |-- resume.pdf
|   |   '-- coverletter.pdf
|   '-- Riverbend Public Library/
|       '-- interview-notes.txt
|-- Certifications/
|-- References/
|-- Resume Library/
'-- _app/
    |-- api.py
    |-- db.py
    |-- build_index.py
    |-- classify.py
    '-- frontend/index.html
```

The tracker root is determined automatically: the application uses the parent directory of `_app/`. There is normally no path you need to enter into a settings screen.

2.2 What the indexer ignores

The filesystem walker deliberately skips the application's own `_app/` directory, hidden files and folders, common virtual environments, dependency folders, archives, and database files. This prevents the tracker from turning its own implementation or backups into job documents.

2.3 What counts as an application

Under `Applications/`, the standard grouping is based on the folder hierarchy. In the common case, the company folder is the company and the next role folder becomes the role label. That means

you can organize documents naturally instead of entering every document into a separate form.

3 First Build: What to Expect

When you click **Build index**, JobTracker walks the tracker root and rebuilds the disposable index from the files currently on disk.

Rebuild is not a reset of your personal tracking data. `jobtracker.db` is regenerated. `overrides.db` is intentionally preserved so notes, manual status changes, dates, snoozes, archives, and company merges survive an index rebuild.

A first rebuild is the point where you should check whether your folder names and filenames are being classified the way you expect.

3.1 Classification is intentionally simple

JobTracker primarily uses filename and folder heuristics rather than opening every PDF and trying to understand its prose. This keeps indexing fast and predictable.

Typical filename signals include:

Filename signal	Likely document type
resume	Resume
cover letter, coverletter	Cover letter
interview, phone screen, screener, schedule	Interview / scheduling material
cheat sheet, interview prep, mock, study	Interview preparation
reject, not referred	Rejection / non-referral
thank you, application received, confirmation	Application confirmation
job bulletin, job description, job id	Job posting

3.2 Customize classification with one file

The primary configuration file is `_app/classify_config.json`. It contains section rules and an optional nested-application pattern for compliance folders.

This is the file to customize first. The generic classifier in `classify.py` is intended to work out of the box and generally should not need editing unless your filename conventions are unusual.

After changing classification rules, rebuild the index. Your `overrides.db` is not rewritten by that rebuild.

4 The Web Dashboard

The web UI is the recommended interface because it exposes the full workflow in one place.

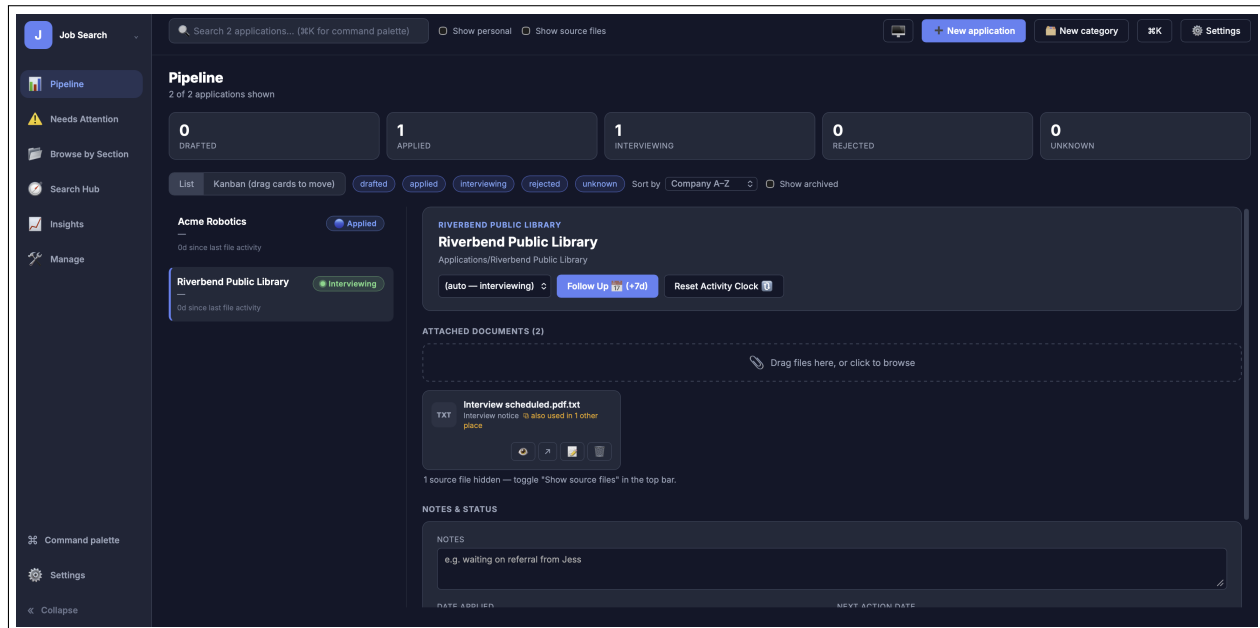
4.1 Pipeline

The **Pipeline** is the main application-management view. It supports both a list representation and a drag-and-drop Kanban board grouped by status.

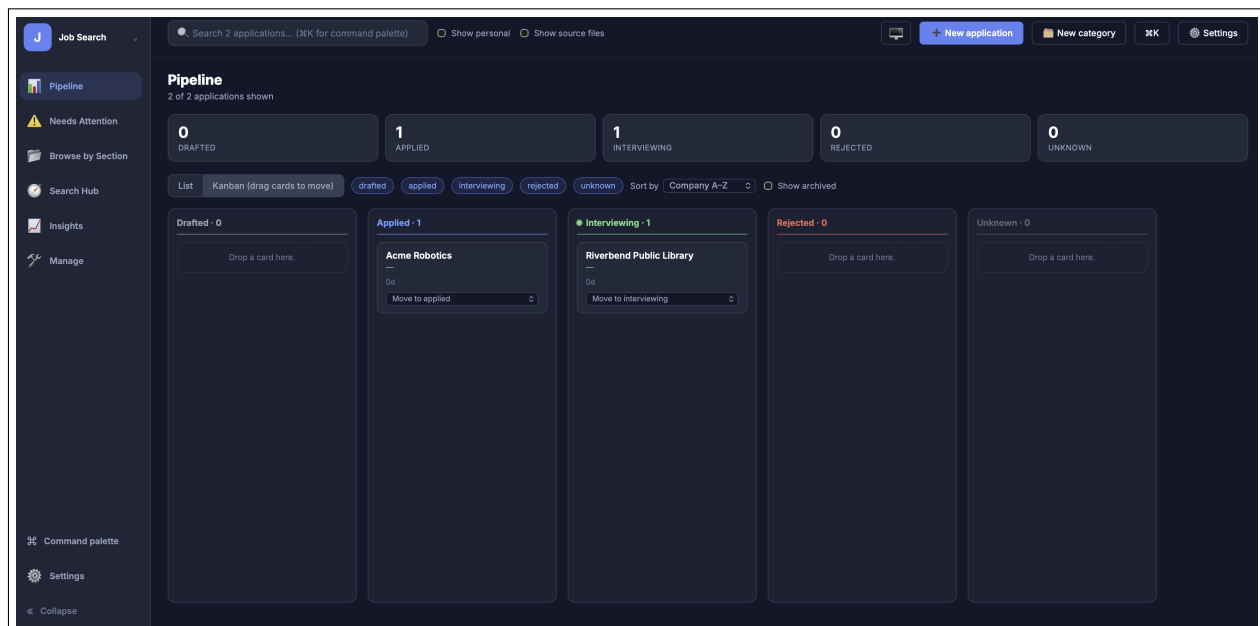
A typical application card can surface:

- company and role information,
- how long the application has been sitting,
- staleness / follow-up indicators,
- attached documents,
- status and manual overrides,
- application dates, notes, and next actions.

Dropping a card into a different Kanban column updates the status immediately.



The Pipeline in list view: status counts, filters, and the detail pane for the selected application.



The same Pipeline in Kanban view — drag a card between columns, or use each card's status dropdown.

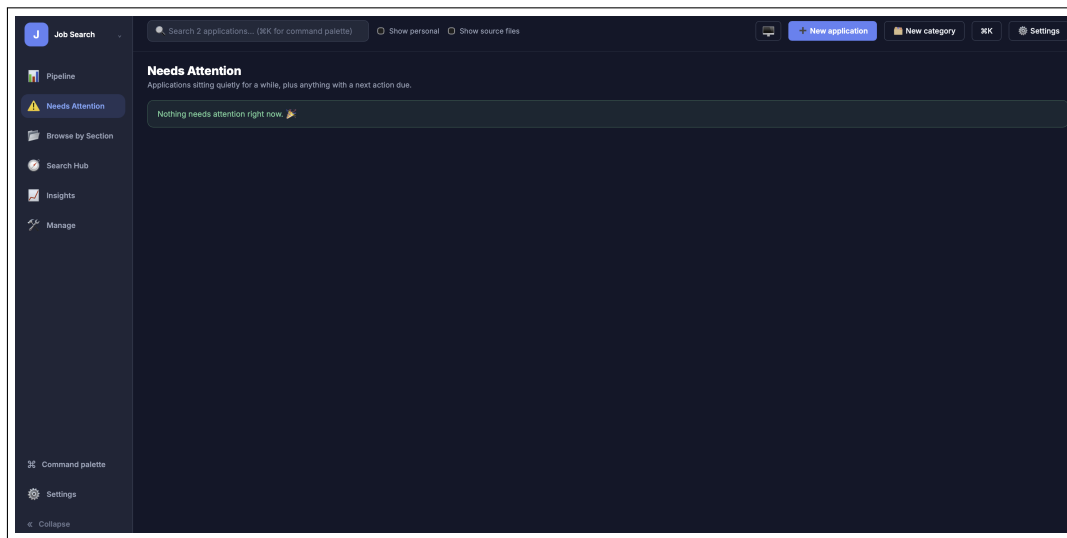
4.2 Needs Attention

Needs Attention is the triage queue. It identifies applications that have gone quiet and applications whose next-action date has arrived.

The default staleness thresholds in the current implementation are:

- applied or interviewing: 21+ days without an update,
- drafted or unknown: 14+ days without an update.

From this view you can mark follow-up work, snooze an item for seven days, archive it, or perform supported bulk actions. Archive is intentionally non-destructive: it changes tracking state rather than deleting your files.



Needs Attention with an empty queue — the state you want to see most mornings.

4.3 Search

Search covers indexed filename, company, role, and full-text search data exposed by the backend. Matching terms are highlighted in the results as you type.

This is particularly useful when the folder tree is large and you remember a company, title, keyword, or document name but not its exact location.

4.4 Browse

Everything outside **Applications/** can be surfaced as configurable sections, such as credentials, network, resume library, leads, compliance, personal material, or your own categories.

The classifier creates a stable section identifier from custom top-level folder names when no explicit mapping exists.

4.5 Insights

Insights turns the indexed application history into a small set of useful metrics, including:

- response rate,
- interview rate,
- average time to response,
- application velocity over time,
- status distribution.

Tip: set a real **Date applied** on active applications. File modification time can change when documents are copied, synced, or reorganized, while Date applied preserves the meaning of the application timeline and improves the velocity analysis.

4.6 Manage

Manage handles higher-level housekeeping:

- merge companies that were split across multiple folder names,
- undo a merge,

- review archived applications,
- restore archived items.

Company merges affect the application's logical view; they do not require you to rename the underlying files.

5 Working With Documents

5.1 Upload

The web app allows documents to be dragged onto an application. New files are written into the application's folder and become visible after the index is updated.

5.2 Rename

Documents can be renamed in place through the web interface. This changes the real file on disk, not merely a label in the database.

5.3 Delete safely

Delete operations are designed around the operating system Trash rather than a permanent unlink. The current implementation also exposes an undo affordance before the trash move is committed.

Still treat delete as a real filesystem operation. The app's safety behavior is better than a hard delete, but the file you are working with is your actual document. Verify the target before confirming a destructive action.

5.4 Source files

.tex source files are indexed and searchable, but they are hidden from normal document lists by default. Enable **Show source files** when you want to work with LaTeX sources alongside their generated PDFs.

5.5 Inline previews

The web app can open supported PDFs in a side drawer and can preview DOCX content through the backend's document-preview path. Text-based sources are also surfaced where appropriate.

6 Command Palette and Keyboard Workflow

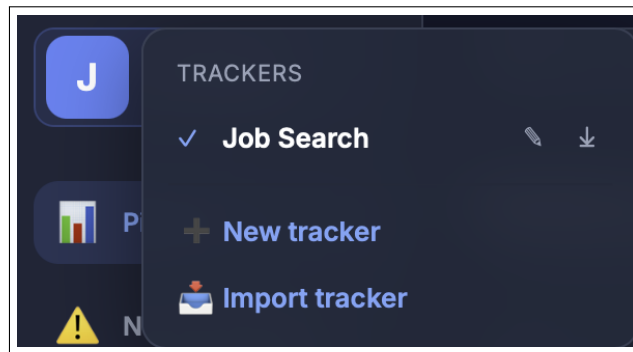
The web UI includes a command palette:

Shortcut	Action
Cmd+K / Ctrl+K	Open the command palette
j / k	Move focus in list views
Arrow keys	Move through list items
Enter	Open the first PDF inline
Esc	Close the active drawer / view

The command palette also exposes view jumps, application search, and the rebuild action.

7 Multiple Trackers and Workspaces

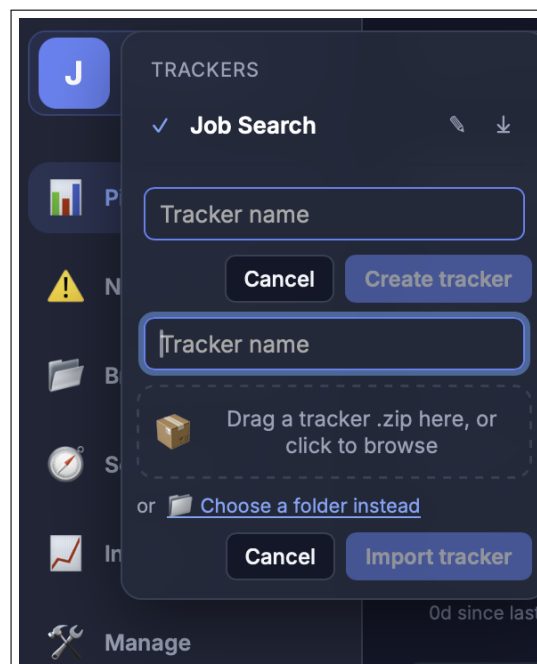
A single running JobTracker instance can manage more than one tracker through its workspace switcher — click the **J** logo, top-left, to open it. Every tracker you have is listed there, with a checkmark on the active one; clicking a different row switches to it instantly. Switching does not start a second server or duplicate the running application — it repoints the one running instance at a different data folder and database pair.



The tracker switcher. Each row has rename (pencil) and export (download) actions; every tracker but the first can also be deleted.

7.1 Creating and importing trackers

New tracker creates a fresh, empty tracker as a sibling folder next to your original one. **Import tracker** brings in an existing folder of documents as a new tracker, two ways: drag a **.zip** onto the dropzone (or click to browse to one), or click **Choose a folder instead** to import directly from a folder picker. Both paths land the same way and apply the same filtering — the application’s own **_app/** directory is never carried into the new tracker, even if it happens to be present in what you’re importing.



The New tracker / Import tracker panel, with both zip and folder import options.

7.2 Exporting and deleting a tracker

Export (the download-arrow icon on a tracker’s row) zips up that tracker’s data — **Applications/**, **Certifications/**, **References/**, and so on — and downloads it, always excluding **_app/** itself. This is the supported way to back up a tracker or hand its data to a genuinely separate installation.

Delete removes a tracker from the switcher *and* sends its folder to the operating system Trash — recoverable there, not a hard, permanent delete. The very first tracker (the one **_app/** is physically nested inside — see Section 2) is the one exception: it can’t be deleted this way, since removing it would mean removing the application’s own home. Rename it instead if you want to repurpose it.

Keep the distinction clear: a workspace is a logical tracker inside the running application; a separate installation is a separate copy of the application code and its local databases, made by copying `_app/` into another tracker root.

8 Privacy and Local Data

JobTracker Hub is designed around local execution.

8.1 Network surface

The API is configured to bind to `127.0.0.1/localhost`. The web frontend talks to that local server, and CORS is restricted to localhost origins plus the `file://` origin used when opening the static frontend directly.

This architecture means the repository itself does not require a hosted backend, login, or cloud account.

8.2 What is stored locally

File	Purpose	Lifecycle
<code>jobtracker.db</code>	Indexed filesystem/application data	Rebuilt from disk whenever you rebuild the index.
<code>overrides.db</code>	Your manual tracking state	Durable; rebuilds deliberately preserve it.
<code>workspaces.json</code> / workspace state	Multi-tracker configuration	Local application state.

These files are gitignored in the repository because they can contain personal job-search data.

9 A Recommended Daily Workflow

The software becomes most useful when the filesystem and the tracking metadata have complementary jobs.

1. **Keep documents organized naturally.** Put the resume, cover letter, job posting, interview notes, and confirmation material for an application under the same company/role area.
2. **Rebuild after structural changes.** If you add or move a large batch of files in Finder/Explorer, rebuild the index so the dashboard reflects the filesystem.
3. **Record the meaningful dates.** Set Date applied rather than relying on file modification dates.
4. **Use Needs Attention as the morning queue.** Handle due follow-ups and stale applications first.
5. **Keep source documents on disk.** Let JobTracker add structure around them instead of replacing your existing document-management habits.
6. **Archive instead of deleting.** Use archive for old applications you want out of the active workflow while retaining their history.

10 Troubleshooting

10.1 The desktop app won't open, or quits immediately

The desktop build keeps a log of its startup backend and, if it can't start, shows a dialog explaining why instead of quitting silently.

1. If a dialog appears, read it — it names the problem and points to the log file directly.

2. If nothing appears at all, open **JobTracker Hub**, click the **J** logo, choose **Settings**, and expand **Diagnostics**. This shows the port, state directory, and log file path for the current session, plus a **Reveal log file** button and a **Copy diagnostic report** button for sharing what's wrong.
3. Relaunch the app once. A single failed launch is occasionally a slow-machine timing issue; a repeated failure is worth checking the log for.

10.2 The dashboard is empty

Confirm that:

1. `_app/` is nested exactly one level inside the tracker root.
2. The tracker root contains at least one real, non-ignored document.
3. You clicked **Build index** after starting the app.

10.3 A folder appears in the wrong section

Edit `_app/classify_config.json`, adjust the section rule, and rebuild. Do not rewrite `classify.py` unless you are changing filename-level document-type heuristics.

10.4 My notes or status changed unexpectedly after a rebuild

A rebuild is not supposed to erase `overrides.db`. Check that the file still exists in the active tracker and that you have not created or switched to a different workspace.

10.5 The browser cannot reach the API

Verify that the terminal running Uvicorn is still active and that it is listening on port 8000. A minimal command is:

```
uvicorn api:app --reload --port 8000
```

Then refresh the frontend.

10.6 A document is missing

First check whether it is hidden because it is a source file such as `.tex`. Then rebuild the index. Also check whether the filename, folder, or extension matches one of the ignored patterns documented in `classify.py`.

11 Repository Map for Contributors

The current repository keeps the implementation intentionally small and readable:

File / directory	Role
<code>_app/api.py</code>	FastAPI backend and local file/API operations.
<code>_app/db.py</code>	Data access, effective status/company state, staleness, and Insights calculations.
<code>_app/build_index.py</code>	Filesystem parser and disposable index builder.
<code>_app/overrides_store.py</code>	Durable user-owned tracking overrides.
<code>_app/classify.py</code>	Filename and section heuristics.
<code>_app/classify_config.json</code>	User-editable folder-to-section rules.
<code>_app/frontend/index.html</code>	Single-file React-based web frontend, loaded from a CDN.
<code>_app/workspace.py</code>	Multiple-tracker/workspace bookkeeping.
<code>_app/labels.py</code>	Section and document-type display labels used by the frontend.
<code>sample-tracker/</code>	Safe fake data for first-run exploration.

12 Repository Safety Notes

JobTracker can manipulate real files, so a few conventions are intentionally strict:

- Path resolution in file endpoints is constrained to the tracker root.
- The indexer skips its own application directory and common generated artifacts.
- Database files are gitignored and should not be committed as part of normal development.
- The sample tracker uses fake organizations and placeholder content so you can test without exposing real applications.

13 License

JobTracker Hub is distributed under the MIT License. See the repository's `LICENSE` file for the authoritative license text.

Source and Design Notes

This guide was written against the repository snapshot supplied with the project on August 25, 2026. Feature descriptions are based on the repository README and the implementation in `_app/`. The visual direction was adapted from Overleaf's *Technical Document Template*, an example software-manual template published in the Overleaf template gallery.¹

The guide deliberately describes the behavior that the current code establishes rather than carrying forward features from an earlier draft. In particular, the application is a single FastAPI web app (Streamlit has been retired — see *Revision 1.2* below) with multiple workspaces/trackers (with per-tracker export and Trash-based delete), Needs Attention triage, local document operations, configurable folder classification, and local SQLite state.

Revision 1.1 adds screenshots of the Pipeline (list and Kanban), Needs Attention, and the tracker switcher, and rewrites Section 7 to match the switcher's current export and delete behavior.

Revision 1.2 removes the Streamlit interface from this guide. The repository's `app.py`, `ui.py`, and `views_*.py` were deleted; the two label dicts the web app actually depended on moved to a new, dependency-free `labels.py`. The web app was already the more capable interface (see Section 4) and never needed Streamlit to run — `api.py` serves `frontend/index.html` itself, so *running it* is now a single command and a single address rather than a choice between two interfaces.

Revision 1.3 adds Section 3.2, **Installing the macOS desktop app**, covering the `.dmg` drag-to-Applications install flow and first-launch folder picker, and adds a Troubleshooting entry for the desktop app's startup log and the new Settings > Diagnostics panel.

Revision 1.4 documents the **Learn More** and **User Guide** shortcuts added to the installer window below the main drag-and-drop row, and redraws the installer-window figure with higher-contrast labels.

Revision 1.5 corrects the installer-window figure and `package-dmg.sh` icon positions: the **Optional extras** row was drawn 40px lower than where Finder actually places those icons, causing the row-2 icons to overlap the caption above them on a real build. Verified against a screenshot of an actual install.

Revision 1.6 rewrites the Gatekeeper warning box in Section 3.2 to match the actual first-launch dialog (titled "Not Opened," offering only **Move to Trash** and **Done** — no **Open Anyway** button), replacing the previous "App is damaged" wording. Adds the explicit **Done**, then right-click **Open**, then **System Settings > Privacy & Security > Open Anyway** fallback sequence, verified against screenshots of an actual blocked launch.

¹Overleaf, *Technical Document Template*, <https://www.overleaf.com/latex/templates/technical-document-template/mdgftpdfbvbs>.